

SmartGN - Digital GN Service Management System

Official Viva & Presentation Preparation Guide

Comprehensive Q&A covering Frontend, Backend, MySQL CRUD, Security & Examiner Trap Questions

SECTION 1: PROJECT OVERVIEW & ELEVATOR PITCH

Q1: How would you summarize SmartGN in 30 seconds for the viva panel?

SmartGN is a web-based Digital Grama Niladhari Service Management System designed to digitize local administration in Sri Lanka. It connects three primary user roles: Residents, GN Officers, and System Administrators. Key modules include digital certificate issuance, appointment booking, disaster reporting, allowance management, and division messaging. The tech stack comprises React 19 & Tailwind CSS v4 on the frontend, Node.js & Express.js for the REST API backend, and MySQL (using connection pooling) for data persistence.

Q2: What key architectural model does the project follow?

The project follows a decoupled client-server architecture with an MVC / Route-Controller-Middleware pattern on the backend:

- Frontend (Client): Single Page Application (SPA) built with React 19 and React Router v7.
- Backend (API Server): Node.js & Express.js REST API providing stateless endpoints.
- Database Layer: MySQL database accessed asynchronously via mysql2/promise connection pool.
- Authentication Layer: Stateless JWT tokens with Role-Based Access Control (RBAC) middleware.

SECTION 2: FRONTEND TECHNOLOGIES (React 19, Vite, Tailwind)

Q3: Why did you use React 19 for the frontend?

React 19 provides component-based architecture and declarative UI rendering via its Virtual DOM. Component state and props make building complex interactive interfaces (like appointment scheduling and certificate approval dashboards) modular, reusable, and easy to maintain.

Q4: What is Vite 8 and why is it preferred over Create React App (CRA)?

Vite 8 is a next-generation frontend build tool. During development, Vite leverages native ES Modules (ESM) to serve code instantly without bundling, enabling lightning-fast Hot Module Replacement (HMR). For production builds, Vite uses Rollup to create highly optimized, minified bundles.

Q5: How is client-side navigation managed without page reloads?

We used React Router v7 (react-router-dom). Client-side routing intercepts browser URL changes and dynamically updates the DOM by rendering matching React components without sending a new HTTP request for an HTML page.

Q6: How is user state and authentication managed across the frontend app?

We implemented the React Context API (AuthContext). It wraps the application root (App.jsx) and supplies global user state (user, token, role) and login/logout handlers to any child component without prop-drilling. The JWT is saved in localStorage to persist user sessions across browser reloads.

Q7: How does the frontend communicate with the backend REST API?

The frontend uses asynchronous HTTP calls (fetch / axios) to communicate with Express routes. For protected endpoints, the frontend retrieves the JWT token from localStorage and attaches it to the Authorization HTTP header.

```
// Example Frontend API Request with JWT
const response = await fetch("http://localhost:5000/api/appointments", {
  method: "GET",
  headers: {
    "Authorization": `Bearer ${token}`,
    "Content-Type": "application/json"
  }
});
const data = await response.json();
```

SECTION 3: BACKEND TECHNOLOGIES (Node.js, Express.js, Middleware)

Q8: Why did you choose Node.js & Express.js for the backend?

Node.js uses an event-driven, non-blocking I/O single-threaded architecture (via libuv event loop), making it extremely fast for handling concurrent HTTP requests and database operations. Express.js is a minimal framework that allows robust route definition and middleware handling.

Q9: How are backend files organized in SmartGN?

The backend source code in Backend/src follows a clean separation of concerns:

- /routes: Route definitions mapping HTTP methods & URIs to controllers.
- /controllers: Business logic and SQL database query execution.
- /middleware: Security checks (JWT verification, role authorization, error handling).
- /config: Database connection pooling and CORS configuration.

Q10: What is Express Middleware and how is it used in SmartGN?

Middleware functions are functions that have access to the request object (req), response object (res), and the next middleware function in the application cycle. We use custom middleware for authentication (authenticateToken), role authorization (requireAdmin, requireOfficerOrAdmin, requireResident), and error handling.

SECTION 4: MYSQL DATABASE & CRUD CODE-BASED QUESTIONS

Q11: How does Node.js connect to MySQL in SmartGN? Single connection vs Connection Pool?

We use mysql2/promise with a Connection Pool (mysql.createPool). A connection pool maintains a reusable group of open database connections. Instead of overhead-heavy opening/closing of TCP connections for every request, requests borrow a connection from the pool and release it when complete. This improves backend performance and handles high concurrent traffic.

Q12: How do you perform CREATE (INSERT) operations in your MySQL controllers?

CREATE operations use parameterized INSERT INTO queries. Example from appointment creation:

```
// INSERT Query Example (appointmentController.js)
const query = `
  INSERT INTO appointment_pending
  (appointment_number, resident_nic, division_id, appointment_date, appointment_time,
  reason)
  VALUES (?, ?, ?, ?, ?, ?)
`;
await db.query(query, [aptNumber, nic, divisionId, date, time, reason]);
```

Q13: How do you perform READ (SELECT) operations with JOINS in SmartGN?

READ operations retrieve data, often joining multiple tables like appointment_pending and resident:

```
// SELECT Query with JOIN Example (appointmentController.js)
const query = `
  SELECT a.*, r.full_name, r.contact_number
  FROM appointment_pending a
  JOIN resident r ON a.resident_nic = r.nic
  WHERE a.division_id = ?
`;
const [appointments] = await db.query(query, [divisionId]);
```

Q14: How do you perform UPDATE operations in MySQL?

UPDATE queries modify existing records based on unique identifiers:

```
// UPDATE Query Example
const query = `
  UPDATE appointment_pending
  SET status = ?, updated_at = NOW()
  WHERE appointment_id = ?
`;
await db.query(query, ["APPROVED", appointmentId]);
```

Q15: How do you perform DELETE operations in MySQL?

DELETE queries remove specific records using primary keys and authorization parameters:

```
// DELETE Query Example
const query = "DELETE FROM appointment_pending WHERE appointment_id = ? AND resident_nic = ?";
await db.query(query, [appointmentId, nic]);
```

Q16: How do you prevent SQL Injection attacks in SmartGN? (Crucial Viva Question)

We use Parameterized Queries (Prepared Statements) supplied by mysql2. We NEVER concatenate raw user input into SQL strings (e.g. "SELECT * FROM user WHERE nic = " + input). Instead, we use ? placeholders. The MySQL engine compiles the SQL command structure first and treats user parameters strictly as data values, neutralizing SQL injection vectors.

Q17: What primary keys and indexing strategies did you use?

We used UUIDs (VARCHAR(36)) for tables like gn_division and household, and NIC (National Identity Card) strings for residents. We added indexes on frequently searched columns (such as division_code, district, and resident_nic) to optimize query lookup speed.

SECTION 5: SECURITY, AUTHENTICATION & ROLE-BASED ACCESS CONTROL

Q18: How are passwords stored in the database?

Passwords are NEVER stored in plain text. We use bcryptjs to compute a salted hash of passwords with 10 salt rounds before saving to MySQL during registration. On login, bcrypt.compare() compares the plaintext input against the stored hash.

Q19: Explain the JWT (JSON Web Token) authentication workflow in SmartGN.

1. User submits login credentials (NIC/Email & Password).
2. Backend verifies credentials and signs a JWT containing user ID and role.
3. Frontend receives JWT and stores it in localStorage.
4. For protected routes, frontend sends "Authorization: Bearer <token>".
5. Middleware verifies token using jwt.verify(token, JWT_SECRET) and attaches user payload to req.user.

Q20: How does Role-Based Access Control (RBAC) work in your Express routes?

RBAC ensures users only access resources authorized for their role (ADMIN, OFFICER, RESIDENT). Express routes chain authorization middleware:

```
// RBAC Middleware Example (Backend/src/middleware/auth.js)
const requireOfficerOrAdmin = (req, res, next) => {
  if (!req.user || (req.user.role !== "ADMIN" && req.user.role !== "OFFICER")) {
    return res.status(403).json({ error: "Access forbidden. Officer/Admin required." });
  }
  next();
};

// Protected Route Usage:
router.get("/division-requests", authenticateToken, requireOfficerOrAdmin, getRequests);
```

Q21: What is CORS and how did you configure it?

Cross-Origin Resource Sharing (CORS) is a browser security feature blocking requests from different origins. We configured cors middleware on Express to explicitly authorize our frontend domain to access backend endpoints.

SECTION 6: EXAMINER TRAP QUESTIONS & IDEAL RESPONSES

Q22: Why did you use MySQL instead of MongoDB (NoSQL)?

SmartGN manages structured, relational municipal data where entities (residents, households, divisions, certificates) have strict relationships and foreign key constraints. MySQL provides ACID compliance, structured transactions, and relational JOIN operations that guarantee data integrity.

Q23: What happens if a user manually changes their role in localStorage to ADMIN?

Role changes in local storage do NOT compromise security. State in local storage is purely for UI rendering. When an API call is made, the backend ignores local storage and inspects the cryptographically signed JWT token. Tampered tokens fail signature validation with `jwt.verify()`, resulting in an HTTP 401/403 error.

Q24: How does your application prevent database credentials or internal stack traces from leaking?

We use centralized error handler middleware (`errorHandler.js`). In production mode, unexpected database errors are logged internally to server logs, while a sanitized, generic error message (e.g. 500 Internal Server Error) is returned to the user.

Q25: What is the difference between `db.query()` and `db.execute()` in mysql2?

`db.execute()` uses binary protocol prepared statements prepared directly on the MySQL server, whereas `db.query()` sends SQL text with parameter replacement. Both support parameterized ? inputs to safeguard against SQL injection.

